

AI 编译器课程大纲

一、课程的特点和优势

在人工智能与边缘计算全面落地的今天，AI 编译器已经成为连接 AI 模型与 NPU/TPU 硬件算力的核心枢纽。无论是大模型的推理加速，还是端侧设备的实时响应，最终都要靠编译器将算法“翻译”成硬件能理解的指令。可以说，编译器的能力直接决定了芯片的效能能否被真正释放——这是芯片产业落地中最关键、也最容易被忽视的卡脖子技术。

随着国产 AI 芯片大规模普及，企业对既懂芯片架构、又精通模型编译的复合型人才需求呈现爆发式增长。然而，市面上大多数课程要么停留在纯理论讲解，要么只是教某个工具的使用方法，距离企业真实岗位需求仍有很大差距。本课程正是为解决这一断层而生——我们从 AI 编译器的完整架构出发，覆盖前端、中端、后端全流程，结合真实 NPU 硬件，打造一门真正意义上的“从理论到落地”的实战课程。

课程的第一个层次，是建立对 AI 编译器架构的系统认知。

我们首先带你梳理当前主流的 AI 编译器方案。TVM 凭借自动调优和跨平台支持成为热门开源选择；TensorFlow XLA 擅长算子融合但局限于谷歌生态；TensorRT 性能极致却与 NVIDIA 硬件深度绑定。这些方案各有优势，但背后暴露了一个核心痛点——IR (中间表示) 的碎片化。前端有 Relay、后端有 LLVM，每开发一款新 NPU，芯片厂商几乎都要从图优化到代码生成“重复造轮子”。这正是为什么我们需要深入理解编译器架构的本质，而不是只会使用某一个工具。

课程的第二个层次，是掌握下一代编译器基础设施——MLIR。

MLIR (Multi-Level Intermediate Representation) 的出现为解决 IR 碎片化提供了全新思路。它不是一个单一的编译器，而是一个构建编译器的“基础设施框架”。其独创的 Dialect (方言) 机制允许为不同硬件抽象层级定义专属表达方式，将复杂的模型到芯片映射拆解为多个可控的渐进式转换步骤 (Lowering)。同时，MLIR 提供了统一的 Pass Manager 和模式重写机制，让算子融合、常量折叠等通用优化可以直接复用社区已有成果，开发者只需专注于编写针对特定 NPU 硬件的专属优化 Pass。这正是目前主流国产 AI 芯片厂商 (如算能、寒武纪等) 不约而同拥抱 MLIR 的根本原因，也是本课程选择 MLIR 作为技术基座的核心考量。

课程的第三个层次，是基于真实 NPU 硬件的全链路实战。

理论最终要落地到硬件。课程依托成熟商业平台工具链，带你完成从模型导入到板端部署的完整闭环。你将亲手操作：ONNX/PyTorch 算子的前端映射、图优化与算子切分、指令调度与内存静态分配、INT8 量化的数学原理与校准算法 (MinMax/KL/Bf16)、性能分析工具的使用与瓶颈定位——每一个环节都有对应的 Lab 让你亲手实践。

课程的第四个层次，是用工业级项目完成能力验证。

实战的终点是能独立搞定真实场景。我们设计了两个贯穿课程的大型项目：一是 YOLOv8 目标检测模型的 NPU 移植，最终实现多路视频流的实时推理，这是安防、智慧交通等领域的标准场景；二是 Llama/Qwen 大语言模型的 NPU 部署，涵盖 Attention 算子加速、算子融合优化、KV Cache 的 NPU 管理机制，最终部署一个可交互的 NPU 加速聊天机器人。这两个项目将前面所有知识点串联起来，让你在真实场景中验证所学。

课程全程依托 IC 修真院专业 AI 算力平台，支持远程访问、分时复用、多人协同学习，真正实现人人可上机、人人可落地、人人可出成果。课程内容由 IC 修真院百位行业资深专家、数十位芯片一线工程师共同打磨，以企业真实岗位需求为导向，确保你学到的是可以直接对接高薪岗位的实战能力。

通过系统化训练，你将彻底掌握 AI 编译器从架构原理到硬件部署的全链路工程方法——这不仅是学会一个工具，更是具备在国产算力爆发时代，快速切入 AI 编译器、边缘 AI 部署、芯片应用开发等高端赛道的核心竞争力。

二、课程适合的学员

- 计算机科学与技术、人工智能、微电子、集成电路、电子信息、自动化、智能科学与技术、物联网工程、通信工程、应用电子等相关理工专业的本科及以上学历毕业生。
- 有 AI 算法开发、模型训练、Python/C++ 开发、嵌入式 Linux 相关经验，希望进一步掌握 AI 编译器、模型优化、边缘部署实战能力，满足企业高薪岗位需求的工程师。
- 有数字芯片、NPU/TPU、FPGA 开发基础，熟悉电路、系统架构等知识，规划向 AI 编译器、模型优化、边缘 AI 部署方向发展的工程师。
- 有一定算法、嵌入式、芯片开发相关基础，希望向 AI 编译器、边缘计算、模型部署等高价值方向进阶发展的工程师。
- 有 AI 开发、嵌入式开发、芯片应用相关经验，想要学习 AI 编译器全流程实战项目技能、提升就业与职场竞争力的工程师。

三、课程培养目标

- 深入理解 AI 编译器全流程架构，熟练掌握模型解析、中间表示 IR、图优化、量化压缩、算子适配、硬件映射、推理部署等核心技术。
- 与项目团队协同工作，具备良好的逻辑思维、工程实现与问题排查能力，能够将业务需求转化为可落地的模型编译与部署方案。
- 熟练使用 AI 编译器工具链完成模型转换、校准、编译、调试与性能分析，确保模型在硬件平台上高效、稳定、低延迟运行。
- 熟悉模型量化、图优化、内存复用、算子融合等优化流程，提升模型在边缘硬件上的算力利用率、帧率与响应速度。
- 掌握编译调试、性能分析、精度修复、异常定位等工程能力，能够独立解决模型部署中的常见问题。
- 掌握 AI 编译器工具使用、命令脚本、环境配置与日志分析，对编译与部署体系具备完整理解与实践认知。
- 编写项目文档、实验报告、部署说明与性能分析记录，确保开发过程可追溯、可复现、可交付。

四、职业规划目标

通过学习 AI 编译器实战课程，直接对口的岗位有：

AI 编译器工程师

- 负责 AI 芯片编译器工具链开发、模型转换、图优化、量化工具、算子适配与硬件映射工作。需求企业：AI 芯片公司、算法框架公司、头部 AI 企业、云计算厂商。
- 边缘 AI 部署工程师
- 主要负责 AI 模型在边缘硬件上的编译优化、工程落地、性能调优与系统集成，将算法转化为可商用的端侧智能应用。需求企业：智能安防、车载 AI、工业视觉、机器人、新能源、物联网企业。
- 模型优化工程师
- 负责模型量化、剪枝、蒸馏、图优化与算子优化，结合 AI 编译器实现端 / 边/云全场景高效部署，是 AI 产业刚需核心岗位。
- AI 芯片应用工程师
- 面向芯片厂商与方案商，负责客户模型适配、编译调试、方案落地与技术支持，属于芯片产业核心技术岗位。

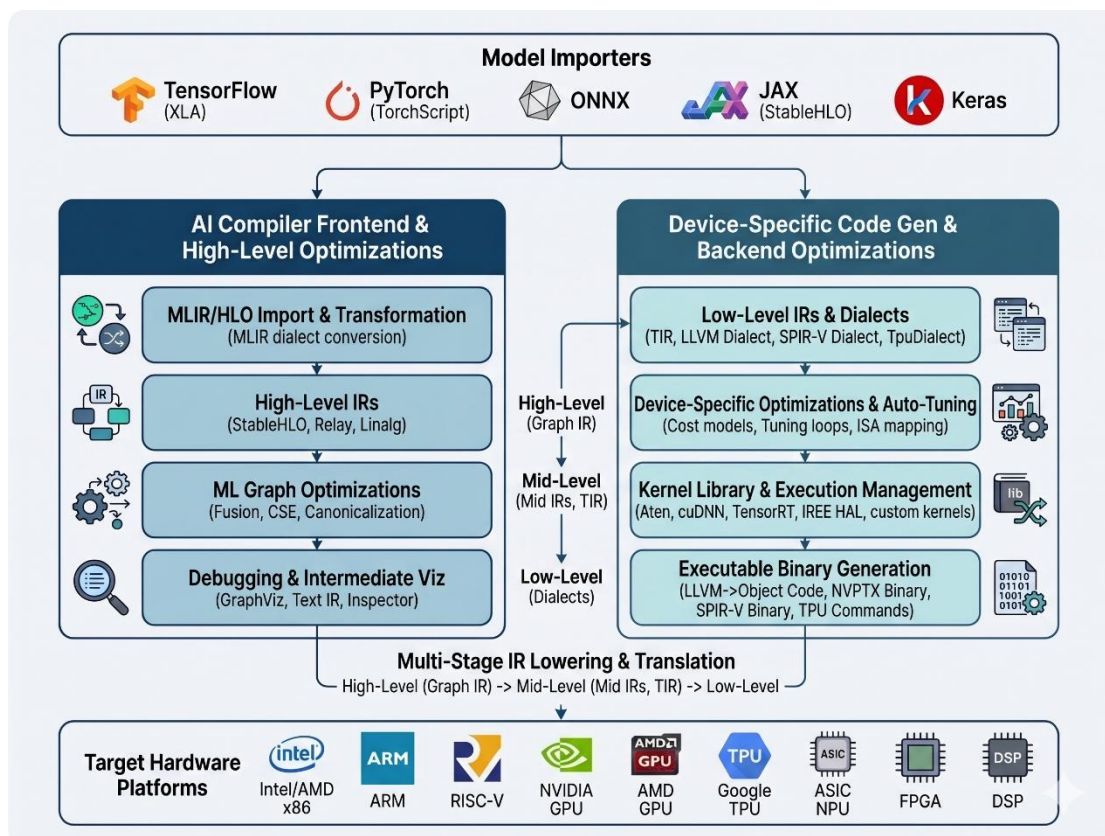
对于技术能力的加深，可以有以下发展方向：

- 技术专家：专注 AI 编译器、模型优化、异构计算、大模型编译等前沿方向，成为领域资深技术专家。
- 项目经理：发展项目管理能力，负责 AI 编译器、模型部署、边缘计算类项目的计划、进度、质量与团队管理。
- 架构工程师：深入理解 AI 编译器与芯片体系架构，发展为编译器架构师、部署架构师、端边云协同架构师。
- 管理职位：向团队负责人、研发主管、技术总监方向发展，负责团队规划、

人才培养与业务落地。

- 创业机会：凭借 AI 编译器与边缘部署核心能力，开展行业解决方案、技术咨询、定制化部署服务等创业方向。

1.1 AI 编译器框架图



1. 模型输入层

- **功能说明：**位于架构图最顶端，代表编译器的输入端。编译器支持解析并接收来自各类算法框架的模型，例如图中标注的 TensorFlow、PyTorch 以及通用的 ONNX 等格式

2. 编译器前端 (Compiler frontend)

- **功能说明：**位于图的左侧区域。前端的核心职责是将不同框架传入的模型统一转换成通用中间表示（IR），并进行初步的优化处理

- **高级 IR (High-level IR / Graph IR):** 图中左侧框图展示了模型被转换为图级别的中间表示。这是一种硬件无关的表达方式，主要用于描述计算逻辑和计算图结构，极大地方便了后续的图优化工
- **计算图优化 (Computation graph Optimizations):** 即前端优化。这是在计算图上执行的与硬件无关的优化步骤，图中清晰列举了具体的优化方法，如 Algebraic simplification (代数化简)、Operator fusion (算子融合)、DCE (对应大纲中的死码消除/无效计算剔除) 等。前端处理完毕后，会输出“优化后的计算图 (Optimized Computation graph)”供后端继续处理。

3. 编译器后端 (Compiler backend)

- **功能说明:** 位于图的右侧区域。后端的任务是把前端传来的优化后的计算图，最终转换为硬件可执行代码，这一阶段包含了大量硬件相关的优化操作。
- **硬件专属优化 (Hardware specific Optimizations):** 针对具体的芯片架构进行性能提升。包含图中列出的 Memory allocation (内存分配)、Memory latency hiding (延迟隐藏)、Loop oriented opt (循环优化) 以及 Parallelization (并行调度与并行化) 等。
- **自动调优 (Auto Scheduling / Auto-tuning):** 通过参数化和成本模型，自动寻找最优的执行策略和参数，以达到最佳性能。
- **调用底层加速库 (Using kernel libraries):** 在编译过程中，会直接调用芯片厂商高度优化的底层算子库，如图中注明的 Intel DNNL、NV cuDNN 等。
- **低级 IR 与代码生成 (Low-level IR & Code generation):** 为了生成具体的指令，后端会将通用表达转换为与硬件强相关的低级 IR，这种 IR 可以描述细粒度的执行逻辑。最终通过代码生成模块，输出针对具体硬件的可执行程序。

4. 目标平台 (Target platforms)

- **功能说明：**位于图的最底部。经过前后端完整的转换与编译后，模型被部署到各类适配的底层硬件平台上，图中涵盖了常见的通用芯片如 CPU、GPU，以及定制化 AI 芯片如 ASIC（包括 TPU 等）。

五、课程大纲

AI 编译器基础（课程时长：8 周） 本课程以通用 NPU 硬件架构与 MLIR 编译器工具链实践（开源工业级框架）为核心平台，沿着“理论认知→工具掌握→技能进阶→项目实战”的主线，通过手写 MLIR 优化 Pass、模型转换全流程跟踪、INT8 量化精度对比、YOLOv8 Stream 实践，以及 Qwen 大语言模型 NPU 部署等六个由浅入深的 Lab 实战，系统性地带你打通从 AI 编译器底层原理到在真实边缘硬件上完成工业级模型高效部署的全链路工程能力。			
序号	课程子阶段	课程内容说明	课程时长
AI 编译器基础			
1	AI 编译器中的 C/C++	Small Vectore/StringRef/ArrayRef, 内存池技术 BumpPtrAllocator, RTTI (isa/cast/dyn_cast), CRTP(静态多态), Visitor, TableGen	3
2	AI 编译器概论 (Graph vs Kernel)	计算图表示, 算子融合理论, AOT vs JIT 编译	2
3	MLIR 基础概念: Dialect/Op/Type	MLIR 语法、中间表达格式、常用的 Builtin Dialect	3
4	MLIR Pass 管理与模式匹配	Pass Manager 工作原理, GreedyPatternRewrite 机制	4
5	Lab1: 手写简单 MLIR 优化 Pass	基于 MLIR 框架编写一个冗余算子消除的 Pass	3
硬件与环境			
1	NPU 硬件架构与存储层次	NPU 核心结构, 计算单元与数据搬运 (DMA) 协同, 局部存储空间管理	4
2	WSL2+Docker 开发环境搭建	配置 WSL2, Docker 映射, NPU 工具链安装	2
3	Lab2: NPU 基础推理 Demo 运行	在 NPU 硬件平台上跑通 ResNet50 的 CPU/NPU 对比实验	3
MLIR 实战剖析			
1	编译器工具链架构与 workflow 概览	模型转换 (Transform) 与部署 (Deploy) 工具链内幕	3
2	Frontend: TopDialect 功能及 converter 介绍	TopDialect 的定义及相关 pass 的功能。如何从 onnx/torch/tflite 转换到 TopDialect	4
3	Backend:	TpuDialect 的设计	4

	TpuDialect 的设计、 conversion/lowering 介绍	Tpu-mlir 里的 conversion: TopToTpu/TopToTosa 简介 Tpumlir 中的 pass 介绍	
4	Tpu-mlir 中的 LayerGroup	layergroup 详解, 基于 layergroup 进行 算子调度/内存编排	4
5	Lab3: 模型转换流程全 跟踪	完成从 ONNX -> MLIR -> BModel 的 全手动转换过程	3
量化与性能优化			
1	INT8 量化数学原理与 误差分析	对称与非对称量化、Per-channel 量化 原理	3
2	校准算法 (MinMax/KL/percentile)	使用校准集寻找最佳阈值、自动寻找 精度敏感层	4
3	Lab4: 量化精度对比与 精度回退	分析量化损失, 使用混合精度提升模 型表现	3
4	性能分析工具 (Profiling) 使用	使用 profiling 工具 定位算子耗时瓶 颈与带宽利用率	2
模型部署实战 (CV)			
1	YOLOv8 目标检测 模型 NPU 部署	在 NPU 硬件平台上部署 yolov8 网络, 实现 yolov8 的预处理(letterbox)与后 处理(N)	4
2	Lab5: YOLOv8 Stream 实践	构建生产者-消费者模型, 实现预处 理、模型推理与后处理的并行化执行	4
LLM 与 Transformer			
1	Attention 算子部署和 优化	Transformer 原理, MHA 部署, Softmax/Matmul 在 NPU 上的并 行指令优化	4
2	KV Cache NPU 管理机 制	解决 LLM 推理过程中的显存瓶颈与 KV 缓存复用	4
3	Qwen3 模型量化与部 署	W8A8 或 W4A16 量化方案在 NPU 平台上的实现	6
4	Lab6: 部署一个 NPU 加速聊天机器人	实现基于 Qwen3 的低延迟 NPU 推 理应用	4
面试辅导			
1	求职辅导	实际的面试场景	4